

Programming Fundamentals in C

Chapter 7: Pointers

Umer Shahid

Department of Electrical Engineering, UET Lahore

November 16, 2025

Today's Agenda

1. Introduction to Pointers
2. Pointer Basics
3. Pointer Operators
4. Pass-by-Reference
5. Using const with Pointers
6. Pointer Arithmetic
7. Pointers and Arrays
8. sizeof Operator
9. Case Study: Bubble Sort
10. Arrays of Pointers
11. Function Pointers
12. Solved Problems
13. Summary and Key Takeaways

What are Pointers?

Definition

A **pointer** is a variable that stores the memory address of another variable.

Regular Variable:

- Directly contains a value
- Example: `int num = 7;`
- Variable `num` holds the value 7

Pointer Variable:

- Contains a memory address
- Points to another variable
- Provides **indirect** access to values

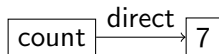
Why Use Pointers?

1. **Pass-by-Reference:** Modify variables in calling function
2. **Dynamic Memory:** Create data structures that grow/shrink
3. **Efficiency:** Pass large data without copying
4. **Arrays & Strings:** Pointers are essential for array manipulation
5. **Function Pointers:** Pass functions as arguments

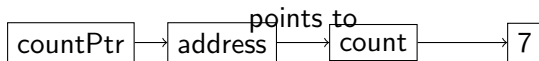
Important

Pointers are one of C's most powerful features, but require careful handling!

Direct vs. Indirect Reference



Direct Reference



Indirect Reference (via pointer)

Indirection

Referencing a value through a pointer is called **indirection**.

Declaring Pointers

Syntax:

```
1 type *pointerName;
```

Examples:

```
1 int *countPtr;      // pointer to an integer
2 double *pricePtr;   // pointer to a double
3 char *letterPtr;     // pointer to a character
```

Reading Right-to-Left

`int *countPtr;` reads as:

“countPtr is a pointer to int”

The `*` indicates that the variable is a pointer.

Pointer Naming Conventions

Common Conventions

- **End with Ptr:** `countPtr`, `dataPtr`
- **Start with p:** `pCount`, `pData`
- **Start with p_:** `p_count`, `p_data`

Best Practice

Always use descriptive names that indicate the variable is a pointer.

Common Mistake: Multiple Declarations

Wrong!

```
1 int *countPtr, count;
```

Here, countPtr is a pointer to int, but count is just an int!

Correct Approach

```
1 int *countPtr;    // Declare pointer separately
2 int count;        // Declare regular variable separately
```

This prevents ambiguity and makes code clearer.

Initializing Pointers

Three ways to initialize a pointer:

1. **NULL** (Preferred):

```
1 int *ptr = NULL;
```

2. **0** (Equivalent to NULL):

```
1 int *ptr = 0;
```

3. **Address of a variable:**

```
1 int num = 5;  
2 int *ptr = &num;
```

Important

- NULL is defined in `<stddef.h>`
- A pointer with value NULL points to nothing
- Always initialize pointers to prevent errors!

Self-Check Question 1

True or False?

The definition:

```
1 int *countPtr, count;
```

specifies that countPtr and count are both of type int * (pointers to integer).

Self-Check Question 1

True or False?

The definition:

```
1 int *countPtr, count;
```

specifies that countPtr and count are both of type int * (pointers to integer).

Answer: FALSE

Actually, count is an int, not a pointer to an int. The * applies only to countPtr and does not distribute to other variables in the definition.

Two Essential Pointer Operators

1. Address Operator (&)

- Returns the memory address of its operand
- Unary operator
- Example: `&variable`

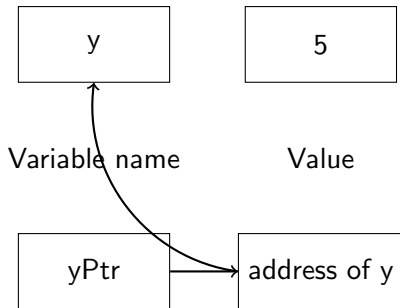
2. Indirection Operator (*)

- Returns the value at the address to which the pointer points
- Also called the **dereferencing operator**
- Unary operator
- Example: `*pointer`

The Address Operator (&)

Purpose: Get the memory address of a variable

```
1 int y = 5;  
2 int *yPtr = &y; // yPtr now holds the address of y
```



Note

The operand of & must be a variable. You cannot use & with constants like 27 or 3.14. 12/81

Memory Representation

```
1 int y = 5;  
2 int *yPtr = &y;
```

Assuming:

- Variable `y` is stored at memory location 600000
- Pointer `yPtr` is stored at memory location 500000

Variable	Location	Value
<code>y</code>	600000	5
<code>yPtr</code>	500000	600000

The Indirection Operator (*)

Purpose: Get the value at the address stored in a pointer

```
1 int y = 5;  
2 int *yPtr = &y;  
3  
4 printf("%d", *yPtr); // Prints 5
```

Dereferencing

Using * in this manner is called **dereferencing a pointer**.
It “follows” the pointer to get the actual value.

Danger!

Dereferencing an uninitialized pointer can:

- Cause a fatal runtime error (crash)
- Corrupt important data
- Create security vulnerabilities

Demonstrating & and * Operators

```
1 #include <stdio.h>
2
3 int main(void) {
4     int a = 7;
5     int *aPtr = &a; // set aPtr to the address of a
6
7     printf("Address of a is %p\n", &a);
8     printf("Value of aPtr is %p\n\n", aPtr);
9
10    printf("Value of a is %d\n", a);
11    printf("Value of *aPtr is %d\n\n", *aPtr);
12
13    printf("&*aPtr = %p\n", &*aPtr);
14    printf("*&aPtr = %p\n", *&aPtr);
15 }
```

Note

%p is used to print pointer values (addresses) in hexadecimal format.

Output Explanation

```
Address of a is 0x7fffe69386cc  
Value of aPtr is 0x7fffe69386cc
```

```
Value of a is 7  
Value of *aPtr is 7
```

```
Showing that * and & are complements of each other  
&*aPtr = 0x7fffe69386cc  
*&aPtr = 0x7fffe69386cc
```

Observations

- Address of a = Value of aPtr (confirming assignment)
- Value of a = Value of *aPtr (dereferencing works)
- &*aPtr and *&aPtr give same result (operators are complements)

Self-Check Question 2

True or False?

Assuming the definitions:

```
1 double d = 98.6;  
2 double *dPtr;
```

The following statement assigns variable d's address to pointer dPtr:

```
1 dPtr = &d;
```

Variable dPtr is then said to “point to” d.

Self-Check Question 2

True or False?

Assuming the definitions:

```
1 double d = 98.6;  
2 double *dPtr;
```

The following statement assigns variable d's address to pointer dPtr:

```
1 dPtr = &d;
```

Variable dPtr is then said to “point to” d.

Answer: TRUE

This is the correct way to make a pointer point to a variable.

Two Ways to Pass Arguments

1. **Pass-by-Value** (Default for non-arrays)
 - A copy of the value is passed
 - Function cannot modify the original variable
 - Used for simple data types
2. **Pass-by-Reference** (Using pointers)
 - The address of the variable is passed
 - Function can modify the original variable
 - More efficient for large data
 - Can “return” multiple values

Pass-by-Value Example

```
1 #include <stdio.h>
2
3 int cubeByValue(int n); // prototype
4
5 int main(void) {
6     int number = 5;
7
8     printf("Original value: %d", number);
9     number = cubeByValue(number); // pass by value
10    printf("\nNew value: %d\n", number);
11 }
12
13 int cubeByValue(int n) {
14     return n * n * n; // cube and return
15 }
```

Output:

```
Original value: 5
New value: 125
```

Pass-by-Reference Example

```
1 #include <stdio.h>
2
3 void cubeByReference(int *nPtr); // prototype
4
5 int main(void) {
6     int number = 5;
7
8     printf("Original value: %d", number);
9     cubeByReference(&number); // pass address
10    printf("\nNew value: %d\n", number);
11 }
12
13 void cubeByReference(int *nPtr) {
14     *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
15 }
```

Output:

```
Original value: 5
New value: 125
```

Pass-by-Value vs Pass-by-Reference

Aspect	Pass-by-Value	Pass-by-Reference
What is passed	Copy of value	Address (pointer)
Function parameter	Regular variable	Pointer variable
Calling syntax	<code>func(x)</code>	<code>func(&x)</code>
Modify original	No	Yes
Multiple returns	No	Yes
Default for	Simple types	Arrays

Principle of Least Privilege

Use pass-by-value unless you explicitly need to modify the caller's variable.

Step-by-Step: Pass-by-Value

1. Before call: `number = 5` in main
2. After call: `n = 5` in function (copy created)
3. Cubing: `n` becomes 125, but `number` unchanged
4. Return: 125 returned to main
5. Assignment: `number = 125`

Step-by-Step: Pass-by-Reference

1. Before call: `number = 5` in main
2. Call establishes: `nPtr` points to `number`
3. Before cubing: `*nPtr = 5` (accessing `number`)
4. After cubing: `*nPtr = 125` (modifies `number` directly)
5. After return: `number = 125` in main

Key Point

The function modifies the caller's variable directly through the pointer!

Important Notes on Pass-by-Reference

Function Parameter Must Be a Pointer

```
1 void cubeByReference(int *nPtr) {  
2     // parameter is pointer to int  
3 }
```

Function Prototype

```
1 void cubeByReference(int *nPtr);  
2 // or simply (parameter name optional):  
3 void cubeByReference(int *);
```

Arrays Are Different

Arrays are automatically passed by reference. You don't use & when passing arrays.

```
1 int arr[10];  
2 someFunction(arr); // NOT &arr
```

The const Qualifier

Purpose

The `const` qualifier tells the compiler that a variable's value should not be modified.

Benefits:

- Enforces principle of least privilege
- Prevents accidental modifications
- Reduces debugging time
- Makes code more robust
- Compiler catches errors

Software Engineering

Always give a function the minimum access it needs—nothing more!

Four Ways to Use const with Pointers

1. **Non-constant pointer to non-constant data**

- Data can be modified through pointer
- Pointer can point to other data

2. **Non-constant pointer to constant data**

- Data cannot be modified through pointer
- Pointer can point to other data

3. **Constant pointer to non-constant data**

- Data can be modified through pointer
- Pointer always points to same location

4. **Constant pointer to constant data**

- Data cannot be modified through pointer
- Pointer always points to same location

Case 1: Non-const Pointer to Non-const Data

```
1 void convertToUppercase(char *sPtr) {  
2     while (*sPtr != '\0') {  
3         *sPtr = toupper(*sPtr); // modify data: OK  
4         ++sPtr;                // modify pointer: OK  
5     }  
6 }
```

Characteristics

- **Highest access level**
- Can modify data through dereferenced pointer
- Can change pointer to point elsewhere
- Use when you need full flexibility

Case 2: Non-const Pointer to Const Data

```
1 void printCharacters(const char *sPtr) {  
2     for (; *sPtr != '\0'; ++sPtr) {  
3         printf("%c", *sPtr);           // read data: OK  
4         // *sPtr = 'X';                 // modify data: ERROR!  
5         ++sPtr;                         // modify pointer: OK  
6     }  
7 }
```

Characteristics

- Data is **read-only**
- Pointer can move to other locations
- Use for functions that only read data
- Enforces principle of least privilege

Read Right-to-Left

`const char *sPtr` means: “sPtr is a pointer to a character constant”

Attempting to Modify Const Data

```
1 void f(const int *xPtr) {  
2     *xPtr = 100;    // ERROR: cannot modify const object  
3 }
```

Compiler Error (Visual C++):

```
error C2166: l-value specifies const object
```

Compiler Error (GCC):

```
error: assignment of read-only location '*xPtr'
```

Important

Different compilers give different error messages, but all will catch this error at compile time!

Case 3: Const Pointer to Non-const Data

```
1 int x = 0;
2 int y = 0;
3
4 int * const ptr = &x;  // must initialize
5
6 *ptr = 7;  // modify data: OK
7 ptr = &y;  // modify pointer: ERROR!
```

Characteristics

- Pointer **always points to same location**
- Data at that location can be modified
- This is how array names work!
- Must be initialized when defined

Read Right-to-Left

`int * const ptr` means: “ptr is a constant pointer to int”

Case 4: Const Pointer to Const Data

```
1 int x = 5;
2 int y = 0;
3 const int * const ptr = &x; // must initialize
4 printf("%d\n", *ptr); // read data: OK
5 *ptr = 7; // modify data: ERROR!
6 ptr = &y; // modify pointer: ERROR!
```

Characteristics

- **Least access level**
- Pointer always points to same location
- Data cannot be modified
- Maximum safety

Read Right-to-Left

`const int * const ptr` means: “ptr is a constant pointer to an integer constant”

Summary: const with Pointers

Declaration	Modify Data?	Modify Pointer?
<code>int *ptr</code>	Yes	Yes
<code>const int *ptr</code>	No	Yes
<code>int * const ptr</code>	Yes	No
<code>const int * const ptr</code>	No	No

Choosing the Right One

Let the **principle of least privilege** guide you:

Give a function only the access it needs to accomplish its task!

Allowed Pointer Arithmetic Operations

1. **Increment/Decrement:** `++ptr`, `--ptr`
2. **Add integer to pointer:** `ptr + n`, `ptr += n`
3. **Subtract integer from pointer:** `ptr - n`, `ptr -= n`
4. **Subtract one pointer from another:** `ptr2 - ptr1`
 - Only meaningful if both point to same array

Warning

Pointer arithmetic on pointers that don't refer to array elements is a logic error!

Example: Integer Array

```
1 int v[5] = {10, 20, 30, 40, 50};  
2 int *vPtr = v; // points to v[0]
```

Assume integers are 4 bytes and array starts at location 3000:

Element	v[0]	v[1]	v[2]	v[3]	v[4]
Value	10	20	30	40	50
Address	3000	3004	3008	3012	3016

Initially: vPtr = 3000 (points to v[0])

Adding Integer to Pointer

```
1 int v[5] = {10, 20, 30, 40, 50};  
2 int *vPtr = v; // vPtr = 3000  
3  
4 vPtr += 2; // What is vPtr now?
```

Adding Integer to Pointer

```
1 int v[5] = {10, 20, 30, 40, 50};  
2 int *vPtr = v; // vPtr = 3000  
3  
4 vPtr += 2; // What is vPtr now?
```

Not Regular Arithmetic!

In regular arithmetic: $3000 + 2 = 3002$

But with pointers:

vPtr becomes $3000 + 2 \times \text{sizeof}(\text{int}) = 3000 + 2 \times 4 = 3008$

Result

vPtr now points to v[2] (address 3008)

*vPtr gives us the value 30

Key Concept: Pointer Arithmetic Scales

General Rule

When you add/subtract an integer n to/from a pointer:

$$\text{new_address} = \text{old_address} \pm (n \times \text{sizeof}(\text{type}))$$

Examples:

- char array (1 byte): `ptr + 3` moves 3 bytes
- int array (4 bytes): `ptr + 3` moves 12 bytes
- double array (8 bytes): `ptr + 3` moves 24 bytes

Why?

The compiler knows the type, so it automatically scales the arithmetic to move by the correct number of bytes!

Incrementing and Decrementing Pointers

```
1 int v[5] = {10, 20, 30, 40, 50};  
2 int *vPtr = v; // points to v[0]  
3  
4 ++vPtr; // now points to v[1]  
5 vPtr++; // now points to v[2]  
6  
7 --vPtr; // back to v[1]  
8 vPtr--; // back to v[0]
```

Prefix vs. Postfix

- ++vPtr and vPtr++ both move pointer forward by one element
- --vPtr and vPtr-- both move pointer backward by one element
- Difference matters only when used in expressions

Subtracting Pointers

```
1 int v[5] = {10, 20, 30, 40, 50};
2 int *vPtr = &v[1];    // points to v[1], address 3004
3 int *v2Ptr = &v[3];    // points to v[3], address 3012
4
5 size_t diff = v2Ptr - vPtr; // diff = ?
```

Subtracting Pointers

```
1 int v[5] = {10, 20, 30, 40, 50};  
2 int *vPtr = &v[1];    // points to v[1], address 3004  
3 int *v2Ptr = &v[3];    // points to v[3], address 3012  
4  
5 size_t diff = v2Ptr - vPtr; // diff = ?
```

Result

diff = 2 (NOT 8!)

The result is the **number of elements** between the pointers, not the number of bytes.

Important

Pointer subtraction is only meaningful when both pointers point to elements of the **same array!**

Comparing Pointers

```
1 int arr[5] = {1, 2, 3, 4, 5};
2 int *p1 = &arr[1];
3 int *p2 = &arr[3];
4
5 if (p2 > p1) {
6     printf("p2 points to higher element\n");
7 }
8
9 if (p1 == NULL) {
10     printf("p1 is NULL\n");
11 }
```

Pointer Comparisons

- Can use: ==, !=, <, >, <=, >=
- Only meaningful for pointers to same array
- Common use: checking if pointer is NULL

Arrays and Pointers: Intimate Relationship

Key Concept

An array name is essentially a **constant pointer** to the array's first element.

```
1 int b[5];  
2 int *bPtr;  
3  
4 bPtr = b;           // equivalent to:  
5 bPtr = &b[0];       // both point to first element
```

Important

- Array name without subscript is a pointer
- But you cannot modify an array name (it's constant)
- Pointers and arrays are largely interchangeable

Four Ways to Access Array Elements

Given:

```
1 int b[5] = {10, 20, 30, 40, 50};  
2 int *bPtr = b;
```

To access element `b[3]`:

1. **Array subscript notation:** `b[3]`
2. **Pointer/offset with array name:** `*(b + 3)`
3. **Pointer subscript notation:** `bPtr[3]`
4. **Pointer/offset with pointer:** `*(bPtr + 3)`

All Four Are Equivalent!

They all access the same memory location and give the same value: 40

Pointer/Offset Notation

```
1 int b[5] = {10, 20, 30, 40, 50};
2 int *bPtr = b;
3
4 // Access b[3] using pointer/offset
5 int value = *(bPtr + 3); // value = 40
6
7 // The 3 is called the "offset"
8 // Parentheses are required!
```

Why Parentheses?

Without parentheses:

```
1 *bPtr + 3    // means: (*bPtr) + 3
2              // = 10 + 3 = 13 (wrong!)
3
4 *(bPtr + 3)  // means: value at address (bPtr + 3)
5              // = 40 (correct!)
```

The * operator has higher precedence than +.

Pointer Subscripting

```
1 int b[5] = {10, 20, 30, 40, 50};
2 int *bPtr = b;
3
4 // You can subscript a pointer like an array!
5 printf("%d\n", bPtr[0]); // prints 10
6 printf("%d\n", bPtr[1]); // prints 20
7 printf("%d\n", bPtr[3]); // prints 40
```

Pointer/Subscript Notation

Once a pointer points to an array, you can use it with subscripts just like the array name.

Flexibility

This allows you to write functions that work with arrays using pointer parameters!

Cannot Modify Array Name

Compilation Error!

```
1 int b[5] = {10, 20, 30, 40, 50};
2
3 b += 3;    // ERROR! Cannot modify array name
4 b++;      // ERROR! Cannot modify array name
```

Correct Approach

```
1 int b[5] = {10, 20, 30, 40, 50};
2 int *bPtr = b;
3
4 bPtr += 3;    // OK! Modifies pointer
5 bPtr++;      // OK! Modifies pointer
```

Reason

Complete Example: All Four Methods

```
1 #include <stdio.h>
2
3 int main(void) {
4     int b[] = {10, 20, 30, 40};
5     int *bPtr = b;
6
7     // Method 1: Array subscript
8     printf("b[2] = %d\n", b[2]);
9
10    // Method 2: Pointer/offset with array name
11    printf("*(b + 2) = %d\n", *(b + 2));
12
13    // Method 3: Pointer subscript
14    printf("bPtr[2] = %d\n", bPtr[2]);
15
16    // Method 4: Pointer/offset with pointer
17    printf("*(bPtr + 2) = %d\n", *(bPtr + 2));
18 }
```

All four print: 30

The sizeof Operator

Purpose

Returns the size (in bytes) of a variable, type, or value.

Syntax:

```
1 sizeof(type)
2 sizeof expression
3 sizeof(variable)
```

Examples:

```
1 sizeof(int)           // size of int type
2 sizeof(double)        // size of double type
3 sizeof x              // size of variable x
4 sizeof(array)         // size of entire array
```

Note

- Returns value of type `size_t`
- Evaluated at compile time (no runtime overhead)

sizeof with Arrays

```
1 float array[20];  
2  
3 printf("Size of array: %zu bytes\n", sizeof(array));
```

If float is 4 bytes:

- `sizeof(array)` returns $20 \times 4 = 80$ bytes

Calculate number of elements:

```
1 double real[22];  
2  
3 size_t numElements = sizeof(real) / sizeof(real[0]);  
4 // = 176 bytes / 8 bytes = 22 elements
```

Formula

$$\text{Number of elements} = \frac{\text{sizeof(array)}}{\text{sizeof(first element)}}$$

sizeof with Pointers vs. Arrays

```
1 #include <stdio.h>
2 #define SIZE 20
3
4 size_t getSize(const float *ptr);
5
6 int main(void) {
7     float array[SIZE];
8
9     printf("sizeof(array) = %zu\n", sizeof(array));
10    printf("getSize(array) = %zu\n", getSize(array));
11 }
12
13 size_t getSize(const float *ptr) {
14     return sizeof(ptr); // size of pointer, not array!
15 }
```

Output (on 64-bit system):

```
sizeof(array) = 80
getSize(array) = 8
```

sizeof: Arrays vs. Pointers

Critical Difference

- `sizeof(array_name)` returns size of **entire array**
- `sizeof(pointer)` returns size of **pointer itself**
- When array is passed to function, it decays to pointer
- Function receives pointer, not array size

Best Practice

When passing arrays to functions, also pass the array size as a separate parameter!

```
1 void processArray(int arr[], size_t size);
```

sizeof with Standard Types

```
1 printf("sizeof(char) = %zu\n", sizeof(char));
2 printf("sizeof(short) = %zu\n", sizeof(short));
3 printf("sizeof(int) = %zu\n", sizeof(int));
4 printf("sizeof(long) = %zu\n", sizeof(long));
5 printf("sizeof(long long) = %zu\n", sizeof(long long));
6 printf("sizeof(float) = %zu\n", sizeof(float));
7 printf("sizeof(double) = %zu\n", sizeof(double));
```

Typical Output (64-bit system):

```
sizeof(char) = 1
sizeof(short) = 2
sizeof(int) = 4
sizeof(long) = 8
sizeof(long long) = 8
sizeof(float) = 4
sizeof(double) = 8
```

Platform Dependent

These sizes can vary across different systems and compilers!

Bubble Sort with Pass-by-Reference

Goal: Improve bubble sort by using pass-by-reference

Two functions:

1. bubbleSort: Sorts the array
2. swap: Exchanges two array elements

```
1 void bubbleSort(int * const array, size_t size);  
2 void swap(int *element1Ptr, int *element2Ptr);
```

Why Pass-by-Reference?

swap needs to modify the actual array elements in the caller, not copies.

The swap Function

```
1 void swap(int *element1Ptr, int *element2Ptr) {  
2     int hold = *element1Ptr;  
3     *element1Ptr = *element2Ptr;  
4     *element2Ptr = hold;  
5 }
```

How it works:

1. Store value at element1Ptr in temporary hold
2. Copy value at element2Ptr to element1Ptr
3. Copy hold to element2Ptr

Key Point

Using *element1Ptr and *element2Ptr, we directly modify the caller's array elements!

Calling swap

```
1 void bubbleSort(int * const array, size_t size) {  
2     for (int pass = 0; pass < size - 1; ++pass) {  
3         for (size_t j = 0; j < size - 1; ++j) {  
4             if (array[j] > array[j + 1]) {  
5                 // Pass ADDRESSES of elements to swap  
6                 swap(&array[j], &array[j + 1]);  
7             }  
8         }  
9     }  
10 }
```

Using Address Operator

`&array[j]` gets the address of element at index `j`.

This enables pass-by-reference for individual array elements.

Complete Bubble Sort Program

```
1 #include <stdio.h>
2 #define SIZE 10
3
4 void bubbleSort(int * const array, size_t size);
5 void swap(int *element1Ptr, int *element2Ptr);
6
7 int main(void) {
8     int a[SIZE] = {2, 6, 4, 8, 10, 12, 89, 68, 45, 37};
9
10    printf("Data in original order:\n");
11    for (size_t i = 0; i < SIZE; ++i) {
12        printf("%4d", a[i]);
13    }
14
15    bubbleSort(a, SIZE);
16
17    printf("\nData in ascending order:\n");
18    for (size_t i = 0; i < SIZE; ++i) {
19        printf("%4d", a[i]);
20    }
```

Output

Data in original order:

2 6 4 8 10 12 89 68 45 37

Data in ascending order:

2 4 6 8 10 12 37 45 68 89

Benefits of This Approach

- swap is reusable
- Code is more modular
- Easier to understand and maintain
- Demonstrates pass-by-reference clearly

Arrays of Pointers

Definition

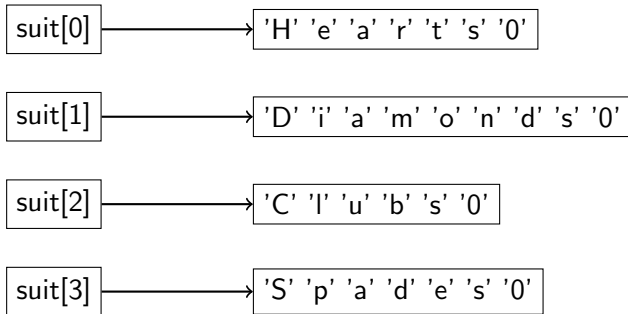
An array where each element is a pointer.

Common use: String arrays

```
1 const char *suit[4] = {  
2     "Hearts",  
3     "Diamonds",  
4     "Clubs",  
5     "Spades"  
6 };
```

- suit is an array of 4 pointers
- Each pointer points to a char (first char of string)
- Strings can be different lengths
- More memory efficient than 2D array

Memory Layout: Array of Pointers



Key Points

- Each array element stores an address (pointer)
- Actual strings stored elsewhere in memory
- Strings can have different lengths

Using Array of Pointers

```
1  const char *suit[4] = {
2      "Hearts", "Diamonds", "Clubs", "Spades"
3  };
4
5  // Print all suits
6  for (int i = 0; i < 4; i++) {
7      printf("%s\n", suit[i]);
8  }
9
10 // Access individual characters
11 printf("First char of Hearts: %c\n", suit[0][0]);
12 printf("Second char of Clubs: %c\n", suit[2][1]);
```

Output:

```
Hearts
Diamonds
Clubs
Spades
First char of Hearts: H
Second char of Clubs: l
```

Array of Pointers vs. 2D Array

2D Character Array:

- Fixed size for each row
- Wastes memory for short strings
- All in contiguous memory

```
1 char suit[4][10] = {  
2     "Hearts",  
3     "Diamonds",  
4     "Clubs",  
5     "Spades"  
6 };
```

Array of Pointers:

- Variable length strings
- No wasted memory
- More flexible

```
1 const char *suit[4] = {  
2     "Hearts",  
3     "Diamonds",  
4     "Clubs",  
5     "Spades"  
6 };
```

Efficiency

Array of pointers is more memory efficient for strings of varying lengths!

Introduction to Function Pointers

Concept

Just as an array name is the address of its first element, a **function name** is the starting address of the code that performs the function's task.

What you can do with function pointers:

- Pass functions as arguments to other functions
- Return functions from functions
- Store functions in arrays
- Assign function pointers to other function pointers
- Compare function pointers

Common Use

Menu-driven systems where user choice determines which function to call.

Declaring Function Pointers

Syntax:

```
1 return_type (*pointer_name)(parameter_types);
```

Examples:

```
1 int (*comparePtr)(int, int);  
2 void (*funcPtr)(void);  
3 double (*calcPtr)(double, double);
```

Read Carefully

`int (*comparePtr)(int, int)` means:

“comparePtr is a pointer to a function that takes two ints and returns an int”

Parentheses around `*comparePtr` are required!

Without Parentheses

Wrong!

```
1 int *compare(int, int);
```

This declares a function that:

- Takes two ints as parameters
- Returns a pointer to int

This is NOT a function pointer!

Correct

```
1 int (*compare)(int, int);
```

This declares a pointer to a function that:

- Takes two ints as parameters
- Returns an int

Example: Sorting with Function Pointers

```
1 // Function prototypes
2 int ascending(int a, int b);
3 int descending(int a, int b);
4 void bubbleSort(int work[], size_t size,
5                 int (*compare)(int, int));
6
7 // Comparison functions
8 int ascending(int a, int b) {
9     return b < a;    // swap if b < a
10 }
11
12 int descending(int a, int b) {
13     return b > a;    // swap if b > a
14 }
```

Flexibility

By passing different comparison functions, we can sort in different orders without rewriting bubbleSort!

Using Function Pointers

```
1 int main(void) {  
2     int a[10] = {2, 6, 4, 8, 10, 12, 89, 68, 45, 37};  
3     int order;  
4     printf("Enter 1 for ascending, 2 for descending: ");  
5     scanf("%d", &order);  
6     if (order == 1) {  
7         bubbleSort(a, 10, ascending);  
8     } else {  
9         bubbleSort(a, 10, descending);  
10    }  
11    // Print sorted array...  
12 }
```

Key Point

ascending and descending are function names, which are automatically converted to pointers when passed as arguments.

Calling Through Function Pointer

Two ways to call:

```
1 void bubbleSort(int work[], size_t size,  
2                 int (*compare)(int, int)) {  
3     // Method 1: Explicit dereference  
4     if ((*compare)(work[i], work[i+1])) {  
5         // swap  
6     }  
7     // Method 2: Implicit dereference  
8     if (compare(work[i], work[i+1])) {  
9         // swap  
10    }  
11 }
```

Both Are Valid

- Method 1 explicitly shows it's a function pointer
- Method 2 looks like a regular function call
- Both produce the same result

Array of Function Pointers

```
1 void function1(int a);
2 void function2(int b);
3 void function3(int c);
4 int main(void) {
5     // Array of 3 function pointers
6     void (*f[3])(int) = {function1, function2, function3};
7     int choice;
8     printf("Enter 0, 1, or 2: ");
9     scanf("%d", &choice);
10    // Call selected function
11    (*f[choice])(choice);
12 }
```

Menu-Driven Systems

This technique is perfect for implementing menus where user choice determines which function to execute!

Problem 1: Understanding Pointer Basics

Question: What does this program print?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int x = 10;
5     int *ptr = &x;
6
7     printf("%d\n", x);
8     printf("%d\n", *ptr);
9
10    *ptr = 20;
11    printf("%d\n", x);
12 }
```

Problem 1: Understanding Pointer Basics

Question: What does this program print?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int x = 10;
5     int *ptr = &x;
6
7     printf("%d\n", x);
8     printf("%d\n", *ptr);
9
10    *ptr = 20;
11    printf("%d\n", x);
12 }
```

Answer:

```
10
10
20
```

Explanation: ptr points to x, so *ptr accesses x's value. Changing *ptr changes x.

Problem 2: Pass-by-Reference

Question: Write a function that swaps two integers using pointers.

Problem 2: Pass-by-Reference

Question: Write a function that swaps two integers using pointers.

Solution:

```
1 void swap(int *a, int *b) {
2     int temp = *a;
3     *a = *b;
4     *b = temp;
5 }
6
7 int main(void) {
8     int x = 5, y = 10;
9     printf("Before: x=%d, y=%d\n", x, y);
10
11     swap(&x, &y);
12
13     printf("After: x=%d, y=%d\n", x, y);
14 }
```

Output:

Before: x=5, y=10

Problem 3: Array and Pointer Arithmetic

Question: What does this program print?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[] = {10, 20, 30, 40, 50};
5     int *ptr = arr;
6
7     printf("%d\n", *ptr);
8     printf("%d\n", *(ptr + 2));
9     printf("%d\n", ptr[4]);
10 }
```

Problem 3: Array and Pointer Arithmetic

Question: What does this program print?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[] = {10, 20, 30, 40, 50};
5     int *ptr = arr;
6
7     printf("%d\n", *ptr);
8     printf("%d\n", *(ptr + 2));
9     printf("%d\n", ptr[4]);
10 }
```

Answer:

```
10
30
50
```

Explanation:

- *ptr gives first element (10)
- *(ptr + 2) is third element (30)

Problem 4: const with Pointers

Question: Which of these declarations will compile without errors?

```
1 int x = 10;
2 const int y = 20;
3
4 // A
5 int *ptr1 = &x;
6 *ptr1 = 15;           // OK?
7
8 // B
9 const int *ptr2 = &y;
10 *ptr2 = 25;          // OK?
11
12 // C
13 int * const ptr3 = &x;
14 ptr3 = &y;           // OK?
```

Problem 4: const with Pointers

Question: Which of these declarations will compile without errors?

```
1 int x = 10;
2 const int y = 20;
3
4 // A
5 int *ptr1 = &x;
6 *ptr1 = 15;           // OK?
7
8 // B
9 const int *ptr2 = &y;
10 *ptr2 = 25;          // OK?
11
12 // C
13 int * const ptr3 = &x;
14 ptr3 = &y;            // OK?
```

Answer:

- A: **OK** - Non-const pointer to non-const data
- B: **ERROR** - Cannot modify data through const pointer

Problem 5: sizeof Operator

Question: What will this program print on a 64-bit system where `int` is 4 bytes?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[10];
5     int *ptr = arr;
6
7     printf("%zu\n", sizeof(arr));
8     printf("%zu\n", sizeof(ptr));
9     printf("%zu\n", sizeof(arr) / sizeof(arr[0]));
10 }
```

Problem 5: sizeof Operator

Question: What will this program print on a 64-bit system where `int` is 4 bytes?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[10];
5     int *ptr = arr;
6
7     printf("%zu\n", sizeof(arr));
8     printf("%zu\n", sizeof(ptr));
9     printf("%zu\n", sizeof(arr) / sizeof(arr[0]));
10 }
```

Answer:

40
8
10

Explanation:

- `sizeof(arr)` = $10 \times 4 = 40$ bytes (entire array)

Problem 6: String Manipulation with Pointers

Question: Write a function to find the length of a string using pointers.

Problem 6: String Manipulation with Pointers

Question: Write a function to find the length of a string using pointers.

Solution:

```
1  #include <stdio.h>
2
3  size_t stringLength(const char *s) {
4      size_t count = 0;
5
6      while (*s != '\0') {
7          count++;
8          s++;
9      }
10
11     return count;
12 }
13
14 int main(void) {
15     char str[] = "Hello";
16     printf("Length: %zu\n", stringLength(str));
17 }
```

Problem 7: Pointer Subtraction

Question: What does this program print?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[] = {1, 2, 3, 4, 5};
5     int *p1 = &arr[1];
6     int *p2 = &arr[4];
7
8     printf("%ld\n", p2 - p1);
9 }
```

Problem 7: Pointer Subtraction

Question: What does this program print?

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[] = {1, 2, 3, 4, 5};
5     int *p1 = &arr[1];
6     int *p2 = &arr[4];
7
8     printf("%ld\n", p2 - p1);
9 }
```

Answer:

3

Explanation:

Pointer subtraction gives the number of elements between the pointers.
From `arr[1]` to `arr[4]` is 3 elements.

Problem 8: Array of Strings

Question: Create and print an array of days of the week.

Problem 8: Array of Strings

Question: Create and print an array of days of the week.

Solution:

```
1 #include <stdio.h>
2
3 int main(void) {
4     const char *days[7] = {
5         "Sunday", "Monday", "Tuesday", "Wednesday",
6         "Thursday", "Friday", "Saturday"
7     };
8
9     printf("Days of the week:\n");
10    for (int i = 0; i < 7; i++) {
11        printf("%d. %s\n", i + 1, days[i]);
12    }
13 }
```

Output:

```
Days of the week:
1. Sunday
```

Problem 9: Finding Maximum Using Pointers

Question: Write a function to find the maximum element in an array using pointers.

Problem 9: Finding Maximum Using Pointers

Question: Write a function to find the maximum element in an array using pointers.

Solution:

```
1 #include <stdio.h>
2
3 int findMax(const int *arr, size_t size) {
4     int max = *arr; // First element
5
6     for (size_t i = 1; i < size; i++) {
7         if (*(arr + i) > max) {
8             max = *(arr + i);
9         }
10    }
11
12    return max;
13 }
14
15 int main(void) {
16     int numbers[] = {45, 12, 89, 23, 67};
17     printf("Maximum: %d\n", findMax(numbers, 5));
```

Problem 10: Reversing Array with Pointers

Question: Reverse an array using two pointers.

Problem 10: Reversing Array with Pointers

Question: Reverse an array using two pointers.

Solution:

```
1 void reverseArray(int *arr, size_t size) {
2     int *start = arr;
3     int *end = arr + size - 1;
4
5     while (start < end) {
6         int temp = *start;
7         *start = *end;
8         *end = temp;
9         start++;
10        end--;
11    }
12 }
13
14 int main(void) {
15     int arr[] = {1, 2, 3, 4, 5};
16     reverseArray(arr, 5);
17     // arr is now {5, 4, 3, 2, 1}
```

Chapter 7 Summary

What We Learned:

1. **Pointer Basics**

- Pointers store memory addresses
- Use & to get address, * to dereference

2. **Pass-by-Reference**

- Modify caller's variables
- More efficient for large data

3. **const Qualifier**

- Four combinations with pointers
- Enforces least privilege

4. **Pointer Arithmetic**

- Scales by object size
- Essential for array manipulation

5. **Pointers and Arrays**

- Array name is constant pointer
- Four ways to access elements

Chapter 7 Summary (cont.)

6. **Arrays of Pointers**

- Efficient for string arrays
- Variable-length strings

7. **Function Pointers**

- Pass functions as arguments
- Menu-driven systems

8. **sizeof Operator**

- Determine sizes at compile time
- Different for arrays vs. pointers

9. **Security**

- Always check for NULL
- Use const appropriately
- Validate array bounds

Key Takeaways

Remember

1. Pointers are powerful but require careful handling
2. Always initialize pointers before use
3. Use const to prevent accidental modifications
4. Understand the relationship between pointers and arrays
5. Check for NULL before dereferencing
6. Respect array boundaries
7. Follow the principle of least privilege

Practice!

Pointers become easier with practice. Work through the exercises in the textbook!

Quick Reference: Pointer Operations

Operation	Syntax	Description
Declare	<code>int *ptr;</code>	Pointer to int
Get address	<code>ptr = &x;</code>	ptr points to x
Dereference	<code>*ptr</code>	Value at ptr
Increment	<code>ptr++</code>	Move to next element
Array access	<code>ptr[i]</code>	ith element
Pointer offset	<code>*(ptr + i)</code>	ith element
Subtract	<code>p2 - p1</code>	Elements between
Compare	<code>p1 == p2</code>	Same address?
NULL check	<code>ptr == NULL</code>	Is NULL?

Questions?

Thank you for your attention!